Does Running a Small LLM in the Browser Change Its Answers? A Case Study of Native ONNX and Browser WebGPU Stacks on a Dual-GPU Laptop

Sudip Niroula
Independent Researcher
Nepal

Mandip Pokharel
Independent Researcher
Nepal

***Abstract*—Browser-based language model inference changes the execution stack, numerical representation, and hardware used to generate an answer. Aggregate benchmark accuracy does not establish whether these changes preserve individual outputs. This paper presents an archived-output case study of Qwen2.5-0.5B-Instruct using native Node.js ONNX CPU execution and browser inference through Transformers.js and WebLLM on a Windows 11 laptop with Intel UHD and NVIDIA GeForce RTX 2050 graphics. Fourteen saved run files contain 3,920 evaluations of fixed subsets comprising 200 MMLU questions and 100 GSM8K problems. The analysis separates task success, parsing coverage, agreement between jointly parsed answers, and literal text agreement. Matched requested-dtype comparisons show severe degradation in the tested Transformers.js WebGPU path: GSM8K task success decreases from 31% to 0%–1% for fp16 and from 24% to 0% for q4f16. A browser WebAssembly q4 diagnostic instead matches native q4 text and raw token IDs on all 20 diagnostic items. A scoring audit identifies 39 unparsed MMLU responses in the native fp32 reference and substantial dependence on last-number extraction for GSM8K. Direct comparison of Intel-labelled and NVIDIA-labelled WebLLM runs finds identical decoded text on all 300 items for q4f32_1, compared with 95% MMLU and 46% GSM8K text agreement for q4f16_1. These observations motivate explicit deployment-fidelity testing, but do not establish an isolated browser effect, a diagnosed runtime defect, or cross-device guarantees.**

***Index Terms*—browser inference, WebGPU, WebAssembly, small language models, ONNX Runtime, WebLLM, quantization, output fidelity, reproducibility**

I. Introduction

Client-side language model inference can avoid per-request inference-server computation and can keep prompts local when the application does not transmit them. Transformers.js and WebLLM provide different software paths for running language models in browser applications [3], [4]. Their availability makes deployment on consumer hardware increasingly practical.

Deployment quality, however, is not described completely by throughput or aggregate benchmark accuracy. A developer may evaluate a model through a native runtime and later deploy a browser-compatible representation. The resulting system may change the weight format, compiler, execution provider, numerical precision, device, and generation defaults. A shared model name is therefore insufficient evidence that the two deployments will produce the same outputs.

Prior work has shown that compression can preserve aggregate benchmark accuracy while changing which individual answers are correct [5]. Browser inference research has also identified numerical precision and hardware heterogeneity as important implementation concerns [6], [7]. These observations motivate paired evaluation: the same questions should be compared item by item, rather than interpreting similar aggregate scores as equivalent behaviour.

This study examines Qwen2.5-0.5B-Instruct across native ONNX CPU execution and browser inference through Transformers.js and WebLLM. The testbed is one consumer laptop with integrated Intel graphics and a discrete NVIDIA GPU. The analysis is intentionally scoped as a case study of recorded deployment configurations, rather than a general comparison of browsers and native inference.

Three evaluation dimensions are distinguished:

- **Task success:** whether an extracted answer matches the benchmark target, counting missing extractions as unsuccessful.
- **Answer fidelity:** whether two configurations produce the same extracted answer, separating jointly parsed answers from shared parsing failures.
- **Text fidelity:** whether generated strings are literally identical, with token-sequence comparisons where the records support them.

These dimensions answer different questions. Two configurations can reproduce the same incorrect answer. They can also achieve the same task-success rate while succeeding on different items. Finally, both can fail to produce a parsable answer, creating outcome agreement without a shared substantive answer.

A. Research Questions

The study addresses four questions:

- **RQ1: Matched requested-dtype comparison.** How do browser Transformers.js WebGPU and WebAssembly outputs compare with native CPU outputs when the same ONNX repository identifier and dtype are requested?
- **RQ2: Cross-adapter consistency.** How often do Intel-labelled and NVIDIA-labelled browser runs on the same host produce identical extracted answers and decoded text?

- **RQ3: Reference fidelity.** How do native quantized ONNX and browser WebLLM deployments differ from a native `fp32` reference in task success, answer agreement, and directional correctness transitions?
- **RQ4: Scoring validity.** How do missing extractions, generation budgets, and permissive extraction rules affect interpretation of the observed results?

B. Contributions

The contributions are:

1) A fixed, item-level evaluation protocol with shared JavaScript message construction and answer extraction.
2) Direct matched requested-dtype comparisons and direct cross-adapter comparisons, rather than comparisons only against aggregate reference accuracy.
3) A scoring audit separating parsing coverage, conditional answer agreement, correctness transitions, and literal text agreement.
4) A versioned analysis of 3,920 archived responses, with explicit provenance and causal limitations.

Correctness flips are not presented as a newly discovered phenomenon. The contribution is their application, together with a parsing audit, to the evaluated native and browser deployment paths.

II. Background and Related Work

A. Browser Inference Systems and Numerical Validation

WebLLM provides browser-local inference through a compiler-based WebGPU execution stack [4]. Transformers.js provides a separate deployment path based on ONNX Runtime, supporting CPU execution through WebAssembly and GPU execution through WebGPU [3].

These frameworks should not be treated as interchangeable implementations of an otherwise completely controlled experiment. They may differ in model representation, quantization, compilation, tokenization interfaces, and generation configuration. Comparisons between them can characterize deployment outcomes without identifying the causal contribution of each internal component.

Llamas on the Web studies memory-efficient, performance-portable, multi-precision WebGPU inference across devices and vendors, noting that reduced-precision accumulation can degrade text coherence in models including Qwen2.5 and motivating 32-bit accumulation [6]. Other recent studies validate single WebGPU operator numerics against CPU references across GPU vendors [7] or examine browser-versus-native likelihood fidelity on domain-specific mixture-of-experts models [8]. The present work focuses instead on paired benchmark answers, parsing behaviour, and decoded-text identity under a fixed consumer laptop deployment protocol.

B. Compression and Behavioural Fidelity

Dutta et al.'s *Accuracy is Not All You Need* establishes that aggregate accuracy can conceal substantial changes in item-level correctness under compression [5]. This motivates reporting both directions of correctness change and distinguishing those changes from wrong-to-wrong answer differences.

TABLE I
Evaluation Inventory

Execution group	Files	Items/file	Total
Native ONNX CPU	4	300	1,200
Transformers.js WebGPU	5	300	1,500
WebLLM WebGPU	4	300	1,200
Transformers.js WASM	1	20	20
Total	14	–	3,920

This study adopts that perspective and adds an extraction distinction. When a reference output does not contain a parsable answer, a later correct extraction is a wrong-to-correct task-success transition. It is not necessarily evidence that a substantive wrong answer became correct.

C. Model and Benchmark Scope

The evaluated model belongs to the Qwen2.5 family [9]. Questions are drawn from MMLU [10] and GSM8K [11].

The results apply to fixed subsets under the prompts, generation limits, and extraction rules used here. They are not full-benchmark scores and are not directly comparable with published results using different prompting, option scoring, or generation protocols.

III. Experimental Design

A. Evaluation Inventory

The archive contains fourteen saved run files. Thirteen contain all 300 questions, while a WebAssembly diagnostic contains ten questions from each task. One full NVIDIA-labelled `fp16` WebGPU run repeats an earlier configuration. Thus, the inventory contains 3,920 item evaluations and 28 task-specific rows, not 28 independent deployment configurations [1], [2].

The repeat file is retained in descriptive reporting but excluded from the family of pairwise significance tests. The small WebAssembly diagnostic is also excluded from that family.

B. Hardware and Software

All inference runs were conducted on one Windows laptop. Table II summarizes the recorded inference and analysis environments [2].

A single host reduces differences in host CPU, installed memory, and operating environment. It does not eliminate execution-stack differences or provide replication across independent machines.

The Chrome version is reported as recorded, rather than as an independently verified complete build identifier. The archive similarly records Node.js only at the stated version granularity.

C. Execution Configurations

Native runs use Transformers.js through the ONNX Runtime Node.js CPU path. Four dtypes are requested from

TABLE II
RECORDED INFERENCE AND ANALYSIS ENVIRONMENT

Component	Recorded configuration
Operating system	Windows 11, 64-bit
CPU	Intel Core i5-13420H (8C/12T)
System memory	8 GB RAM
Integrated graphics	Intel UHD Graphics, Gen-12LP
Discrete graphics	NVIDIA GeForce RTX 2050 Laptop GPU, 4 GB GDDR6
Browser	Chrome user-agent reporting 139.0.0.0
Native host	Node.js v20+
Transformers.js	4.3.0
WebLLM	0.2.85
Analysis environment	Python 3.14.2, SciPy 1.18.1, Transformers 5.17.0

`onnx-community/Qwen2.5-0.5B-Instruct`: `fp32`, `fp16`, `q4`, and `q4f16`.

Browser Transformers.js runs use the WebGPU path for `fp16` and `q4f16` under both adapter labels. The WebAssembly diagnostic uses `q4` on the CPU.

Browser WebLLM runs use the `q4f16_1` and `q4f32_1` precompiled builds. In the MLC quantization registry, these builds differ in their `float16` and `float32` `model_dtype` settings [12]. They do not constitute an accumulation-only intervention.

Throughout this paper, a dtype identifies the requested model representation. It does not establish the precision of every intermediate computation or native CPU instruction.

D. Adapter Labelling

Chrome was assigned to the power-saving or high-performance graphics preference in Windows. The browser page recorded adapter information using a main-page `navigator.gpu.requestAdapter()` call [2].

This is evidence about the page-visible adapter after the preference change, not direct attestation of the internal adapter handle selected by every inference worker. Consequently, the terms *Intel-labelled* and *NVIDIA-labelled* are used for the saved browser runs.

The WebAssembly diagnostic executes on the CPU. An available GPU recorded in page metadata does not make it a GPU inference run.

E. Historical Artifact Provenance

The native and browser Transformers.js runners request the same repository identifier and dtype strings. However, historical ONNX revisions and file checksums were not pinned across inference dates [2].

The comparison therefore establishes matched requested identifiers, not verified byte-for-byte identity of every historically downloaded model artifact. This distinction is retained in the interpretation of RQ1.

F. Questions and Generation Protocol

The fixed evaluation file contains 200 MMLU questions and 100 GSM8K problems, sampled with seed 2026. The MMLU subset covers all 57 subjects: 34 subjects contribute four questions, 18 contribute three, and five contribute two [2].

Both runners import `site/harness.js` for message construction and answer extraction. The system message is:

> You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

MMLU generation is limited to five new tokens. GSM8K generation is limited to 320 new tokens. Sampling was disabled (`do_sample = false`) in Transformers.js, and WebLLM used temperature zero and seed zero [2].

These settings remove intentional sampling variation. They do not guarantee identical outputs across hardware, implementations, or sessions.

The shared harness controls application-level message construction. It does not by itself prove that every framework renders identical internal prompt token sequences or applies identical generation defaults.

IV. EVALUATION AND STATISTICAL METHODS

A. Archived Extraction Rules

The archived MMLU extractor first searches for a standalone uppercase A–D letter. If unsuccessful, it falls back to an uppercase A–D character without a word-boundary requirement. The fallback can therefore match a character inside a word [2].

The GSM8K extractor searches for an `answer is <number>` pattern and otherwise returns the last number in the output. Such a number can be a final answer or an intermediate calculation in an incomplete response.

The historical predictions and correctness flags are preserved. Additional extraction audits are reported separately rather than silently replacing the original scores.

B. Task Success and Parsing Coverage

For item i, let g_i denote the target and a_i the extracted answer. A missing extraction is denoted by $\perp$. Task success is

$$S = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{a_i \neq \perp \wedge a_i = g_i\}. \tag{1}$$

Parse rate is

$$P = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{a_i \neq \perp\}. \tag{2}$$

Accuracy among parsed responses divides the number of correct answers by the number of nonmissing extractions. It is diagnostic, not an unbiased replacement for task success, because configurations can produce parsable answers on different subsets.

A strict-extraction audit requires a standalone uppercase A–D letter (`r"\b([ABCD])\b"`) for MMLU and a restrictive explicit marker match (`r"answer is\s*([\-0-9,\.]+)"`) for GSM8K. Strict parse rate (`strict_parse_rate`) measures extraction coverage

under this restrictive regex; strict-extraction accuracy (`strict_acc`) also requires the extracted answer to match the target.

These stricter rules do not guarantee that an extraction is the intended final answer. A standalone letter can occur in explanatory text, while an explicit numeric answer can still be incorrect.

C. Answer Agreement

For reference answer r_i and test answer t_i, define the jointly parsed set

$$B = \{i : r_i \neq \perp \wedge t_i \neq \perp\}. \tag{3}$$

Valid-answer agreement is

$$A_{\text{valid}} = \frac{\sum_{i \in B} \mathbf{1}\{r_i = t_i\}}{|B|}. \tag{4}$$

It is undefined when $|B| = 0$. All-outcome agreement is

$$A_{\text{all}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{r_i = t_i\}, \tag{5}$$

where two missing extractions count as equal outcomes.

Numerators and denominators are reported explicitly when space permits. High valid-answer agreement on a very small jointly parsed subset is not evidence of broad fidelity.

D. Correctness Transitions

Let N_{c2w} count items on which the reference succeeds and the test configuration does not. Let N_{w2c} count the opposite transition. Missing answers are unsuccessful under these definitions.

The net change in task success is

$$\Delta S = \frac{N_{\text{w2c}} - N_{\text{c2w}}}{N}, \tag{6}$$

whereas the correctness-flip rate is

$$F = \frac{N_{\text{w2c}} + N_{\text{c2w}}}{N}. \tag{7}$$

The analysis additionally identifies transitions involving a missing reference or test extraction (`unparsed_ref_to_correct` and `correct_to_unparsed_test`). These transitions remain part of task success but should not all be described as substantive answer reversals.

E. Text and Token Fidelity

Exact text agreement compares literal decoded strings without whitespace trimming. Raw generated token IDs are compared across all paired items when both records contain them.

When raw IDs are unavailable, strings are retokenized using the `Qwen/Qwen2.5-0.5B-Instruct` tokenizer explicitly loaded at pinned revision:

`7ae557604adf67be50417f59c2c2f167def9a775`

The loaded `tokenizer.json` artifact has SHA-256 checksum `c0382117ea329cdf097041132f6d735924b697924d6f6fc3945713e96ce87539` [2].

The first divergent token is the first unequal position, or the shorter sequence length when one sequence is a strict prefix of the other. The sample median is calculated over divergent sequences.

Raw-token divergence and retokenized-text divergence are not interchangeable. Retokenization cannot recover every detail of original generation, including special tokens removed during decoding.

F. Token-Cap Indicators

For runs containing raw output IDs, token-cap indicators use the stored sequence length. For WebLLM records without those IDs, they use retokenized output length [2].

Reaching the nominal generation cap is not identical to a verified truncation finish reason. Retokenized lengths are also not exact reconstructions of the original generation length. The paper therefore reports cap indicators, not exact counts of unfinished solutions.

G. Statistical Analysis

Paired task-success differences are evaluated using a two-sided exact binomial McNemar test, implemented through SciPy's binomial test [13], [14].

For $b = N_{\text{c2w}}$, $c = N_{\text{w2c}}$, and $m = b + c > 0$, the test evaluates discordant outcomes under a binomial null with probability $1/2$. When $m = 0$, the comparison has $p = 1$.

Holm's step-down correction is applied to 22 comparisons against native `fp32`, excluding reference rows, the repeat file, and the WebAssembly diagnostic [2], [15].

All-outcome agreement intervals use 2,000 percentile bootstrap resamples with seed zero. These intervals describe item-resampling variation under the fixed protocol, not uncertainty across hardware, drivers, or sessions. Degenerate intervals at complete or zero observed agreement do not establish population certainty.

All inferential results are exploratory because the study uses small, structured benchmark subsets and lacks independent host-level replication.

V. Results

A. MMLU Parsing Changes the Interpretation

The native `fp32` reference succeeds on 78 of 200 MMLU items and leaves 39 unparsed (38 saturating the five-token limit and one terminating early at three tokens with the option text `WEP` plus EOS on item `mmlu-1554`). Native `q4` and `q4f16` obtain higher task-success scores while producing fewer unparsed outputs. WebLLM configurations also have higher parsing coverage than the reference [2].

Figure 1 separates correct, parsed incorrect, and unparsed outcomes. It illustrates why higher task success should not automatically be interpreted as improved underlying knowledge.

Native `q4f16` has 25 wrong-to-correct transitions relative to `fp32`, but eight originate from missing `fp32` answers. The

TABLE III
MMLU TASK SUCCESS AND PARSING AUDIT. EACH CONFIGURATION CONTAINS 200 ITEMS. CONDITIONAL ACCURACY USES EACH CONFIGURATION'S OWN PARSED SUBSET. STRICT PARSE RATE EQUALS PERMISSIVE PARSE RATE FOR ALL NATIVE AND WEBLLM ROWS SHOWN HERE.

Configuration	Correct	Unparsed	Parse rate (%)	Task success (%)	Correct among parsed
Native fp32	78	39	80.5	39.0	78/161 = 48.45%
Native fp16	76	40	80.0	38.0	76/160 = 47.50%
Native q4	91	14	93.0	45.5	91/186 = 48.92%
Native q4f16	93	15	92.5	46.5	93/185 = 50.27%
WebLLM q4f16_1, Intel-labelled	84	1	99.5	42.0	84/199 = 42.21%
WebLLM q4f16_1, NVIDIA-labelled	86	0	100.0	43.0	86/200 = 43.00%
WebLLM q4f32_1, Intel-labelled	89	6	97.0	44.5	89/194 = 45.88%
WebLLM q4f32_1, NVIDIA-labelled	89	6	97.0	44.5	89/194 = 45.88%

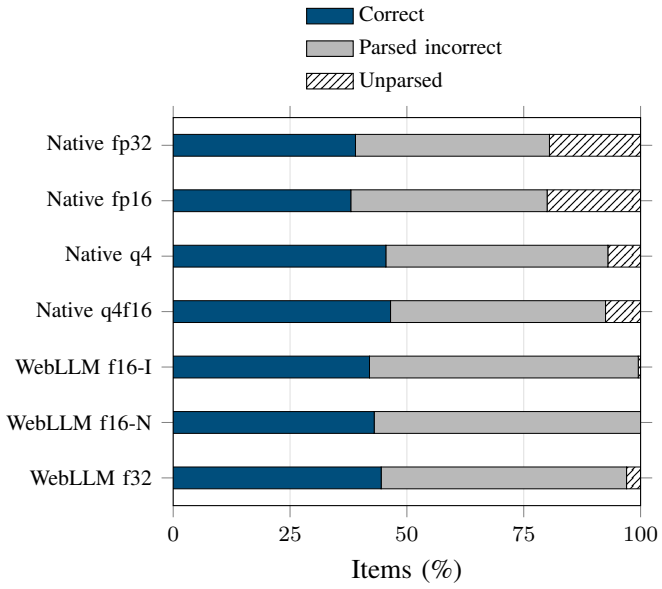


Fig. 1. MMLU outcome decomposition under the five-token generation budget ($N = 200$ per configuration). WebLLM f16-I and f16-N denote the Intel-labelled and NVIDIA-labelled `q4f16_1` runs, respectively. WebLLM f32 represents the identical outcome counts of the two `q4f32_1` runs. Data are from the pinned parsing export [2].

corresponding counts are 11 of 29 for Intel-labelled WebLLM `q4f16_1`, 12 of 30 for its NVIDIA-labelled counterpart, and 10 of 30 for each `q4f32_1` run [2].

Shared missing answers affect agreement as well. Native `fp16` and `fp32` match on 196 of 200 outcomes, but 39 matches are shared parsing failures. Agreement among jointly parsed answers is 157/160, or 98.12%.

The five-token protocol therefore measures a mixture of answer correctness, formatting behaviour, and budget sensitivity. The audit identifies this confound; it does not quantify the counterfactual score under a longer generation budget.

B. GSM8K Scores Depend on Extraction Policy

All native and WebLLM configurations produce a number under the archived GSM8K extractor. Explicit-marker extraction succeeds on very few outputs, however. Native `fp32` and `fp16` have no explicit-marker matches despite archived task-success scores of 29% and 31%, respectively [2].

The archived scores consequently depend heavily on last-number fallback extraction. The strict audit measures extraction-policy sensitivity, not whether every response without an explicit marker is mathematically incorrect.

The cap indicators likewise should not be interpreted as verified unfinished solutions. Without consistently recorded finish reasons, the distinction between completed and budget-limited generation remains imperfect.

C. Matched Requested-Dtype Comparisons

The browser WebAssembly `q4` diagnostic matches native CPU `q4` in decoded text and raw token IDs on all twenty diagnostic items. Their task-success rates on those same items are 20% for MMLU and 40% for GSM8K [2].

In contrast, the tested `fp16` and `q4f16` WebGPU runs show severe degradation relative to native runs with the same requested dtype (Tables V and VI).

Figure 2 visualizes the GSM8K contrast. The zero-valued WebGPU measurements are explicitly marked at the baseline so that measured zero success cannot be mistaken for missing data.

Because the model identifiers were not historically checksum-pinned, the finding is stated as a discrepancy under matched requested identifiers rather than identical verified model bytes.

Relative to matched native `fp16`, the Intel-labelled and NVIDIA-labelled WebGPU `fp16` outputs diverge at the first raw generated token on 87/100 and 82/100 GSM8K items, respectively (Table VI). Relative to matched native `q4f16`, both `q4f16` WebGPU runs diverge at token zero on all 100 GSM8K items (whereas relative to native `fp32` in Appendix A, 99 of 100 items diverge at token zero and 1 item diverges at token one) [2].

The WebAssembly diagnostic supports successful end-to-end parity for its configuration and subset. It does not isolate which component accounts for the `fp16` or `q4f16` WebGPU discrepancy. Possible distinctions include graph processing, operator execution, cache handling, runtime versions, and generation behaviour. The logs do not diagnose one of these as the cause.

TABLE IV
GSM8K EXTRACTION SENSITIVITY AND TOKEN-CAP INDICATORS. EACH CONFIGURATION CONTAINS 100 ITEMS. MARKER PARSED COUNTS MATCHES TO THE RESTRICTIVE EXPLICIT-MARKER REGEX (STRICT_PARSE_RATE); MARKER CORRECT (STRICT_ACC) ALSO REQUIRES THE EXTRACTED NUMBER TO MATCH THE TARGET.

Configuration	Archived correct	Archived parsed	Marker parsed	Marker correct	At cap	Length source
Native fp32	29	100	0	0	44	Raw token IDs
Native fp16	31	100	0	0	41	Raw token IDs
Native q4	20	100	3	1	50	Raw token IDs
Native q4f16	24	100	3	1	50	Raw token IDs
WebLLM q4f16_1, Intel-labelled	16	100	0	0	50	Retokenized text
WebLLM q4f16_1, NVIDIA-labelled	19	100	0	0	49	Retokenized text
WebLLM q4f32_1, Intel-labelled	18	100	2	1	45	Retokenized text
WebLLM q4f32_1, NVIDIA-labelled	18	100	2	1	45	Retokenized text

TABLE V
DIRECT MATCHED REQUESTED-DTYPE COMPARISONS (TASK AND ANSWER OUTCOMES). NATIVE AND BROWSER VALUES USE THE SAME ITEM IDS. THE NVIDIA FP16 REPEAT HAS THE SAME REPORTED SCORES AS ITS FIRST RUN AND IS OMITTED HERE FOR BREVITY.

Browser configuration	Task	N	Native success (%)	Browser success (%)	Valid agreement	All-outcome agreement
WASM q4 diagnostic	MMLU	10	20.0	20.0	8/8 (100.00%)	10/10 (100.00%)
WASM q4 diagnostic	GSM8K	10	40.0	40.0	10/10 (100.00%)	10/10 (100.00%)
TJS WebGPU fp16, Intel-labelled	MMLU	200	38.0	5.5	13/43 (30.23%)	42/200 (21.00%)
TJS WebGPU fp16, NVIDIA-labelled	MMLU	200	38.0	7.0	11/45 (24.44%)	42/200 (21.00%)
TJS WebGPU fp16, Intel-labelled	GSM8K	100	31.0	0.0	0/56 (0.00%)	0/100 (0.00%)
TJS WebGPU fp16, NVIDIA-labelled	GSM8K	100	31.0	1.0	0/51 (0.00%)	0/100 (0.00%)
TJS WebGPU q4f16, Intel-labelled	MMLU	200	46.5	2.5	3/17 (17.65%)	17/200 (8.50%)
TJS WebGPU q4f16, NVIDIA-labelled	MMLU	200	46.5	2.0	1/16 (6.25%)	16/200 (8.00%)
TJS WebGPU q4f16, Intel-labelled	GSM8K	100	24.0	0.0	0/4 (0.00%)	0/100 (0.00%)
TJS WebGPU q4f16, NVIDIA-labelled	GSM8K	100	24.0	0.0	1/5 (20.00%)	1/100 (1.00%)

TABLE VI
DIRECT MATCHED REQUESTED-DTYPE COMPARISONS (GENERATION FIDELITY AGAINST MATCHED-DTYPE NATIVE CONTROLS). EXACT TEXT AND EXACT RAW TOKEN MATCHES ARE REPORTED AS COUNTS ALONGSIDE TOKEN-LEVEL DIVERGENCE STATISTICS.

Browser configuration	Task	N	Exact text	Exact token IDs	Median div. token	Div. at token 0	Div. at token 1
WASM q4 diagnostic	MMLU	10	10/10	10/10	–	0/10	0/10
WASM q4 diagnostic	GSM8K	10	10/10	10/10	–	0/10	0/10
TJS WebGPU fp16, Intel-labelled	MMLU	200	0/200	0/200	0	196/200	3/200
TJS WebGPU fp16, NVIDIA-labelled	MMLU	200	1/200	1/200	0	196/200	2/200
TJS WebGPU fp16, Intel-labelled	GSM8K	100	0/100	0/100	0	87/100	4/100
TJS WebGPU fp16, NVIDIA-labelled	GSM8K	100	0/100	0/100	0	82/100	8/100
TJS WebGPU q4f16, Intel-labelled	MMLU	200	1/200	1/200	0	193/200	6/200
TJS WebGPU q4f16, NVIDIA-labelled	MMLU	200	0/200	0/200	0	193/200	7/200
TJS WebGPU q4f16, Intel-labelled	GSM8K	100	0/100	0/100	0	100/100	0/100
TJS WebGPU q4f16, NVIDIA-labelled	GSM8K	100	0/100	0/100	0	100/100	0/100

D. Direct Cross-Adapter Comparisons

WebLLM `q4f32_1` produces identical decoded text across the two labelled runs on all 300 items. WebLLM `q4f16_1` instead matches on 190/200 MMLU strings and 46/100 GSM8K strings (Table VII) [2].

Figure 3 shows the text-agreement result. It establishes consistency of decoded outputs for this dataset and labelled pair. It does not establish an accumulation-only mechanism or a guarantee across devices.

The Transformers.js `q4f16` GSM8K comparison demonstrates a different issue. Both runs have zero task success but 97% all-outcome agreement. Their valid-answer agreement is based on only three jointly parsed items. Agreement without parsing coverage and task success would therefore be misleading.

E. Comparison Against Native FP32

The complete paired comparisons are provided in Appendix A.

Native `q4` and `q4f16` each have 35 MMLU correctness transitions, or 17.5% of all items. WebLLM `q4f16_1` has 52 transitions, or 26.0%, while `q4f32_1` has 49, or 24.5% [2].

These rates include parsing-related transitions. Nevertheless, answer turnover remains after restricting comparison to jointly parsed responses. Native `q4` and `q4f16` each agree with `fp32` on 121/161 parsed pairs, or 75.16%. WebLLM configurations agree on 61.49%–64.60% of jointly parsed pairs.

TABLE VII
DIRECT INTEL-LABELLED VERSUS NVIDIA-LABELLED COMPARISONS. MMLU CONTAINS 200 ITEMS AND GSM8K CONTAINS 100. ALL-OUTCOME AGREEMENT INCLUDES SHARED MISSING ANSWERS.

Browser build	Task	Intel success (%)	NVIDIA success (%)	Valid agreement	All-outcome agreement	Exact text
WebLLM q4f16_1	MMLU	42.0	43.0	194/199 (97.49%)	194/200 (97.00%)	190/200 (95.00%)
WebLLM q4f16_1	GSM8K	16.0	19.0	59/100 (59.00%)	59/100 (59.00%)	46/100 (46.00%)
WebLLM q4f32_1	MMLU	44.5	44.5	194/194 (100.00%)	200/200 (100.00%)	200/200 (100.00%)
WebLLM q4f32_1	GSM8K	18.0	18.0	100/100 (100.00%)	100/100 (100.00%)	100/100 (100.00%)
Transformers.js fp16	MMLU	5.5	7.0	37/41 (90.24%)	170/200 (85.00%)	128/200 (64.00%)
Transformers.js fp16	GSM8K	0.0	1.0	24/42 (57.14%)	59/100 (59.00%)	27/100 (27.00%)
Transformers.js q4f16	MMLU	2.5	2.0	11/12 (91.67%)	189/200 (94.50%)	140/200 (70.00%)
Transformers.js q4f16	GSM8K	0.0	0.0	3/3 (100.00%)	97/100 (97.00%)	36/100 (36.00%)

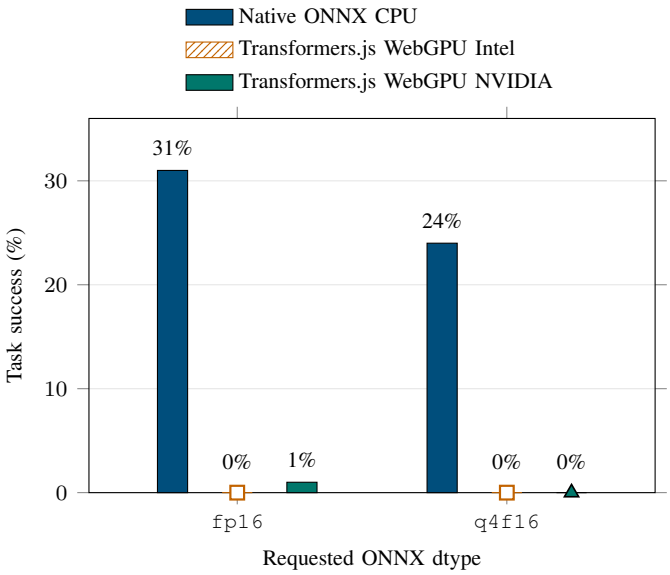


Fig. 2. GSM8K task success in matched requested-dtype comparisons ($N = 100$ per run), using the archived permissive extraction rules. Intel and NVIDIA denote the adapter-labelled runs. Baseline markers and 0% labels represent measured zero task success, not missing observations. The figure compares Transformers.js WebGPU with native ONNX CPU execution; it does not show WebLLM results [2].

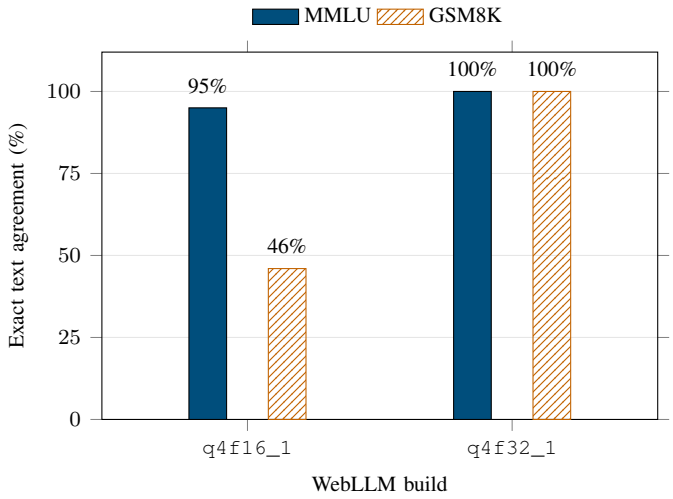


Fig. 3. Direct decoded-text agreement between Intel-labelled and NVIDIA-labelled WebLLM runs. MMLU uses 200 paired items and GSM8K uses 100. Complete observed agreement is limited to these recorded outputs [2].

The result supports the established distinction between net accuracy change and item-level fidelity [5]. It does not imply that every departure from `fp32` is harmful: `fp32` is a reference configuration rather than a ground-truth model.

F. Statistical Interpretation

After Holm correction across 22 comparisons (reported in Appendix B), the eight non-repeat Transformers.js WebGPU task comparisons against `fp32` remain significant at the family-wise 0.05 threshold. None of the native quantized or WebLLM comparisons remains significant under that correction [2].

For example, the WebLLM `q4f32_1` GSM8K comparison has an unadjusted p-value of 0.0266 and a Holm-adjusted p-value of 0.3192.

A non-significant correctness test does not establish output equivalence. On MMLU, `q4f32_1` agrees with `fp32` on only 104/161 jointly parsed answers despite a non-significant net task-success difference.

These tests assess paired correctness, not equality of strings, model distributions, or reasoning processes.

G. Initialization and Repeatability

The diagnostic notes report `fp32` WebGPU initialization failures with `ERROR_CODE: 6` and `std::bad_alloc` under both selected graphics preferences [2].

The available logs do not isolate host-memory allocation, WebAssembly heap allocation, GPU buffers, or another loader component. The observation establishes an initialization failure of this configuration, not a universal inability to run `fp32` models in browsers.

The saved NVIDIA-labelled `fp16` WebGPU repeat reproduces the reported task-success scores and directional flip counts. This supports repeatability of the observed outcome under the repeated configuration, not universal determinism across sessions.

VI. Discussion

A. Consistency and Correctness Are Different

The results distinguish two deployment properties.

First, WebLLM `q4f32_1` is fully consistent across the two labelled runs in decoded text on the tested items, but differs substantially from native `fp32`.

Second, degraded Transformers.js configurations can agree on failed outcomes while achieving little or no task success. A fidelity claim therefore needs both an explicit reference and a description of the compared outcome.

"Consistent across the two browser runs" is different from "faithful to native `fp32`." Neither statement alone establishes answer quality.

B. What the Runtime Comparison Establishes

The matched requested-dtype comparisons provide evidence of a large deployment-path discrepancy. It appears under both adapter labels and affects both task success and generation sequences.

However, moving from native CPU to browser WebGPU changes the execution provider and device, may change runtime builds and graph-processing paths, and lacks historical model checksums. The evidence does not isolate the browser environment as the causal variable.

A root-cause investigation would require a smaller failing example, stronger artifact provenance, controlled prompt-token comparisons, and intermediate or fixed-prefix numerical validation. Those experiments are outside the present archived-output analysis.

C. Interpreting the WebLLM Build Comparison

The `q4f32_1` build shows stronger observed cross-adapter text consistency than `q4f16_1`. This gives a practical reason to evaluate both builds when consistency matters.

It does not demonstrate that accumulation precision alone caused the result. The builds differ in model dtype and compiled numerical behaviour [12]. The study also does not evaluate their latency, memory, energy, or throughput trade-offs.

Thus, `q4f32_1` is a candidate for validation, not a universal deployment recommendation.

D. Scoring Is Part of the System

The five-token MMLU limit mixes knowledge assessment with brevity and format compliance. The GSM8K fallback introduces a different ambiguity: a numeric prediction can originate from an intermediate calculation instead of an explicit final answer.

A stronger follow-up protocol should retain the original scoring for historical comparison while adding:

- longer MMLU generation with separately reported parse rates;
- option-score evaluation where comparable scores are available;
- GSM8K finish reasons and explicit final-answer extraction;
- manual review of a predefined sample of fallback extractions;
- fixed-prefix or teacher-forced numerical comparisons where logits are accessible.

These additions would help distinguish formatting, incomplete generation, numerical differences, and substantive answer changes.

VII. Threats to Validity

A. Internal Validity

Historical model hashes are missing. Matching requested identifiers does not establish identical downloaded bytes. The shared harness does not prove identical framework-level prompt tokenization, and page-level adapter metadata does not attest to each worker's selected device.

The twenty-item `q4` WebAssembly check is not a complete browser CPU baseline for `fp16` or `q4f16`. ONNX-to-WebLLM comparisons also change quantization, representation, compiler, runtime, and execution device together.

These limitations restrict causal attribution, even where the observed output discrepancy is large.

B. Construct Validity

Task success depends on prompts, generation budgets, and extraction rules. The `fp32` configuration is a reference, not a correctness oracle.

Literal text identity is deliberately strict. Harmless wording or whitespace changes can reduce text agreement without reducing usefulness. Conversely, extracted-answer agreement can conceal different reasoning or malformed generation.

Conditional agreement and conditional accuracy are also selection-sensitive because each configuration can produce valid answers on a different subset.

C. Statistical Validity

The main tasks contain only 200 and 100 items. Most configurations have one saved run, preventing reliable estimation of session-to-session variation. The structured MMLU subset is not a simple population-wide sample.

Holm correction addresses the specified family of correctness tests. It does not compensate for absent host-level replication or make this exploratory case study confirmatory.

D. External Validity

The study evaluates one small model on one laptop, one recorded browser environment, and two adapter labels.

Findings should not be generalized to larger models, mobile hardware, other operating systems, other browser engines, or later software versions without additional experiments.

In particular, the severe WebGPU degradation is a result for the tested path, not a permanent property of browser inference.

VIII. Reproducibility and Data Availability

The original benchmark archive containing the unmodified initial evaluation harness and all fourteen raw output JSON files is permanently archived as version 1.0.0 on Zenodo at DOI 10.5281/zenodo.23760946 [1]. The updated static-audit analysis pipeline, pinned tokenizer configuration, and five exported CSV summary tables are archived as version 1.1.0 on Zenodo at DOI 10.5281/zenodo.23828643 and in the GitHub repository [2].

The exact Git commit corresponding to version 1.1.0 and used to generate all reported tables is:

```
f2e35b70009a23e25e72caa63c64494b4395b461
```

The principal analysis command is:

```
python analyze.py results
native-node-cpu/fp32
```

The repository and version 1.1.0 archive export primary comparisons, parsing audits, matched-runtime comparisons, cross-adapter comparisons, and Holm-adjusted tests. The tokenizer revision is explicitly pinned in the analysis [2].

This paper reports analysis of archived outputs. It does not claim that a new independent set of inference runs was performed during manuscript preparation.

Future replications should additionally record model-file checksums, exact dependency artifacts, browser build, graphics-driver versions, worker-level adapter details, prompt token IDs, generation parameters, and finish reasons.

IX. Practical Implications

Deployment testing should evaluate the actual target stack rather than treating successful native execution as sufficient validation of a browser configuration.

A practical validation suite should report task success, parse rate, jointly parsed answer agreement, shared missing outcomes, directional correctness transitions, and text agreement separately. A fixed canary set can detect severe regressions but should complement, rather than replace, larger task evaluation.

For applications prioritizing cross-adapter consistency, `q4f32_1` is a reasonable candidate to test alongside `q4f16_1`. The present finding supports that testing decision, not a guarantee for other devices or workloads.

Finally, allocation failures should be described at the level actually observed. An undiagnosed initialization error does not establish a universal browser memory limit or make quantization universally mandatory.

X. Conclusion

This case study examines output fidelity across native ONNX CPU and browser inference stacks using fixed benchmark items on a dual-GPU consumer laptop.

The archived results show severe degradation in the tested Transformers.js WebGPU path relative to matched requested-dtype native controls, alongside complete text and raw-token parity in a twenty-item browser WebAssembly `q4` diagnostic.

They also show that aggregate task-success scores conceal substantial item-level turnover. Interpretation requires separating valid-answer disagreement from parsing changes and acknowledging the dependence of GSM8K results on fallback extraction.

Direct cross-adapter comparison finds identical decoded text on all 300 items for WebLLM `q4f32_1`, whereas `q4f16_1` differs on ten MMLU and fifty-four GSM8K outputs. This is a scoped empirical consistency result, not an accumulation-only causal finding or a cross-device guarantee.

The methodological implication is that deployment fidelity should be measured directly, with an explicit reference and transparent scoring rules, rather than inferred from model identifiers or aggregate benchmark accuracy.

Author Contributions

Sudip Niroula: conceptualization, methodology, software, evaluation-harness design, audit-pipeline development, formal analysis, and manuscript drafting and revision.

Mandip Pokharel: hardware configuration, execution of native and browser benchmark runs, data curation, and output validation.

Appendix A
Complete FP32 Reference Comparisons

Tables VIII and IX report all 28 task rows against the native `fp32` reference. The WebAssembly diagnostic is compared with `fp32` only on its corresponding ten items per task.

C2W and W2C include transitions involving missing extractions, with transitions from an unparsed reference response to a correct test response (`unparsed_ref_to_correct`) and from a correct reference response to an unparsed test response (`correct_to_unparsed_test`) listed explicitly in Table IX. The p-values in Table VIII are unadjusted.

Appendix B
Holm Step-Down Multiple-Testing Correction

Table X reports the 22-hypothesis Holm step-down multiple-testing correction across the 11 full experimental configurations and two benchmark tasks compared against `native-node-cpu/fp32` (excluding the reference rows, the repeat run, and the 20-item WASM diagnostic check).

TABLE VIII
COMPLETE PAIRED COMPARISON AGAINST NATIVE FP32 (ANSWER OUTCOMES AND PARSING). MMLU ROWS HAVE $N = 200$ AND GSM8K ROWS HAVE $N = 100$, EXCEPT WASM DIAGNOSTIC ROWS, WHICH HAVE $N = 10$. I AND N DENOTE INTEL-LABELLED AND NVIDIA-LABELLED RUNS. ALL VALUES ARE FROM THE PINNED ANALYSIS EXPORT [2].

Configuration	Task	Success (%)	Parse rate (%)	Valid agreement	All-outcome agreement [95% CI]	Unadjusted p
Native fp32	MMLU	39.0	80.5	161/161 (100.00%)	200/200 (100.00%) [1.000, 1.000]	1.0000
Native fp16	MMLU	38.0	80.0	157/160 (98.12%)	196/200 (98.00%) [0.960, 0.995]	0.5000
Native q4	MMLU	45.5	93.0	121/161 (75.16%)	135/200 (67.50%) [0.610, 0.735]	0.0410
Native q4f16	MMLU	46.5	92.5	121/161 (75.16%)	136/200 (68.00%) [0.615, 0.740]	0.0167
WASM q4 diagnostic	MMLU	20.0	80.0	2/7 (28.57%)	4/10 (40.00%) [0.100, 0.700]	1.0000
TJS WebGPU fp16 I	MMLU	5.5	27.0	13/43 (30.23%)	41/200 (20.50%) [0.155, 0.260]	$< 10^{-4}$
TJS WebGPU fp16 N	MMLU	7.0	27.0	11/46 (23.91%)	42/200 (21.00%) [0.155, 0.270]	$< 10^{-4}$
TJS WebGPU fp16 N rep	MMLU	7.0	27.0	11/46 (23.91%)	42/200 (21.00%) [0.155, 0.270]	$< 10^{-4}$
TJS WebGPU q4f16 I	MMLU	2.5	9.0	5/14 (35.71%)	40/200 (20.00%) [0.150, 0.260]	$< 10^{-4}$
TJS WebGPU q4f16 N	MMLU	2.0	8.0	2/13 (15.38%)	38/200 (19.00%) [0.140, 0.250]	$< 10^{-4}$
WebLLM q4f16_1 I	MMLU	42.0	99.5	99/161 (61.49%)	100/200 (50.00%) [0.430, 0.570]	0.4885
WebLLM q4f16_1 N	MMLU	43.0	100.0	100/161 (62.11%)	100/200 (50.00%) [0.430, 0.570]	0.3317
WebLLM q4f32_1 I	MMLU	44.5	97.0	104/161 (64.60%)	110/200 (55.00%) [0.480, 0.620]	0.1524
WebLLM q4f32_1 N	MMLU	44.5	97.0	104/161 (64.60%)	110/200 (55.00%) [0.480, 0.620]	0.1524
Native fp32	GSM8K	29.0	100.0	100/100 (100.00%)	100/100 (100.00%) [1.000, 1.000]	1.0000
Native fp16	GSM8K	31.0	100.0	74/100 (74.00%)	74/100 (74.00%) [0.650, 0.820]	0.6875
Native q4	GSM8K	20.0	100.0	19/100 (19.00%)	19/100 (19.00%) [0.120, 0.270]	0.0490
Native q4f16	GSM8K	24.0	100.0	23/100 (23.00%)	23/100 (23.00%) [0.150, 0.320]	0.3018
WASM q4 diagnostic	GSM8K	40.0	100.0	2/10 (20.00%)	2/10 (20.00%) [0.000, 0.500]	1.0000
TJS WebGPU fp16 I	GSM8K	0.0	56.0	3/56 (5.36%)	3/100 (3.00%) [0.000, 0.070]	$< 10^{-4}$
TJS WebGPU fp16 N	GSM8K	1.0	51.0	2/51 (3.92%)	2/100 (2.00%) [0.000, 0.050]	$< 10^{-4}$
TJS WebGPU fp16 N rep	GSM8K	1.0	51.0	2/51 (3.92%)	2/100 (2.00%) [0.000, 0.050]	$< 10^{-4}$
TJS WebGPU q4f16 I	GSM8K	0.0	4.0	0/4 (0.00%)	0/100 (0.00%) [0.000, 0.000]	$< 10^{-4}$
TJS WebGPU q4f16 N	GSM8K	0.0	5.0	0/5 (0.00%)	0/100 (0.00%) [0.000, 0.000]	$< 10^{-4}$
WebLLM q4f16_1 I	GSM8K	16.0	100.0	15/100 (15.00%)	15/100 (15.00%) [0.080, 0.220]	0.0146
WebLLM q4f16_1 N	GSM8K	19.0	100.0	19/100 (19.00%)	19/100 (19.00%) [0.120, 0.270]	0.0525
WebLLM q4f32_1 I	GSM8K	18.0	100.0	20/100 (20.00%)	20/100 (20.00%) [0.120, 0.290]	0.0266
WebLLM q4f32_1 N	GSM8K	18.0	100.0	20/100 (20.00%)	20/100 (20.00%) [0.120, 0.290]	0.0266

TABLE IX
COMPLETE PAIRED COMPARISON AGAINST NATIVE FP32 (TRANSITIONS AND GENERATION FIDELITY). TOKEN DIVERGENCE USES RAW TOKEN IDS FOR NATIVE AND TRANSFORMERS.JS RUNS AND RETOKENIZED TEXT FOR WEBLLM RUNS [2].

Configuration	Task	C2W / W2C	Unparsed ref→C / C→Unparsed test	Exact text	Med. div. tok.	Div@0 / Div@1
Native fp32	MMLU	0 / 0	0 / 0	200/200	–	0 / 0
Native fp16	MMLU	2 / 0	0 / 0	182/200	1	5 / 12
Native q4	MMLU	11 / 24	7 / 0	97/200	0	66 / 29
Native q4f16	MMLU	10 / 25	8 / 0	95/200	0	65 / 32
WASM q4 diagnostic	MMLU	1 / 0	0 / 0	3/10	0	6 / 1
TJS WebGPU fp16 I	MMLU	73 / 6	2 / 56	0/200	0	196 / 2
TJS WebGPU fp16 N	MMLU	72 / 8	3 / 56	1/200	0	196 / 2
TJS WebGPU fp16 N rep	MMLU	72 / 8	3 / 56	1/200	0	196 / 2
TJS WebGPU q4f16 I	MMLU	77 / 4	1 / 75	2/200	0	179 / 18
TJS WebGPU q4f16 N	MMLU	78 / 4	0 / 74	0/200	0	180 / 19
WebLLM q4f16_1 I	MMLU	23 / 29	11 / 0	70/200	0	100 / 24
WebLLM q4f16_1 N	MMLU	22 / 30	12 / 0	69/200	0	100 / 28
WebLLM q4f32_1 I	MMLU	19 / 30	10 / 0	74/200	0	91 / 30
WebLLM q4f32_1 N	MMLU	19 / 30	10 / 0	74/200	0	91 / 30
Native fp32	GSM8K	0 / 0	0 / 0	100/100	–	0 / 0
Native fp16	GSM8K	2 / 4	0 / 0	54/100	84.5	0 / 3
Native q4	GSM8K	13 / 4	0 / 0	0/100	0	72 / 5
Native q4f16	GSM8K	10 / 5	0 / 0	0/100	0	74 / 7
WASM q4 diagnostic	GSM8K	2 / 2	0 / 0	0/10	0	8 / 1
TJS WebGPU fp16 I	GSM8K	29 / 0	0 / 12	0/100	0	87 / 4
TJS WebGPU fp16 N	GSM8K	29 / 1	0 / 15	0/100	0	82 / 8
TJS WebGPU fp16 N rep	GSM8K	29 / 1	0 / 15	0/100	0	82 / 8
TJS WebGPU q4f16 I	GSM8K	29 / 0	0 / 26	0/100	0	99 / 1
TJS WebGPU q4f16 N	GSM8K	29 / 0	0 / 27	0/100	0	99 / 1
WebLLM q4f16_1 I	GSM8K	19 / 6	0 / 0	0/100	1	0 / 69
WebLLM q4f16_1 N	GSM8K	16 / 6	0 / 0	0/100	1	0 / 69
WebLLM q4f32_1 I	GSM8K	16 / 5	0 / 0	0/100	1	0 / 70
WebLLM q4f32_1 N	GSM8K	16 / 5	0 / 0	0/100	1	0 / 70

TABLE X
HOLM STEP-DOWN MULTIPLE-TESTING CORRECTION ACROSS 22 PRIMARY HYPOTHESES AGAINST NATIVE FP32. I AND N DENOTE INTEL-LABELLED AND NVIDIA-LABELLED RUNS [2].

Configuration	Task	Discordant flips	Unadjusted p	Holm rank	Holm-adjusted p	Significant at $\alpha = 0.05$
TJS WebGPU q4f16 N	MMLU	c2w=78, w2c=4	7.614297×10^{-19}	1	1.675145×10^{-17}	Yes
TJS WebGPU q4f16 I	MMLU	c2w=77, w2c=4	1.449536×10^{-18}	2	3.044026×10^{-17}	Yes
TJS WebGPU fp16 I	MMLU	c2w=73, w2c=6	9.995152×10^{-16}	3	1.999030×10^{-14}	Yes
TJS WebGPU fp16 N	MMLU	c2w=72, w2c=8	5.375096×10^{-14}	4	1.021268×10^{-12}	Yes
TJS WebGPU fp16 I	GSM8K	c2w=29, w2c=0	3.725290×10^{-9}	5	6.705523×10^{-8}	Yes
TJS WebGPU q4f16 I	GSM8K	c2w=29, w2c=0	3.725290×10^{-9}	6	6.705523×10^{-8}	Yes
TJS WebGPU q4f16 N	GSM8K	c2w=29, w2c=0	3.725290×10^{-9}	7	6.705523×10^{-8}	Yes
TJS WebGPU fp16 N	GSM8K	c2w=29, w2c=1	5.774200×10^{-8}	8	8.661300×10^{-7}	Yes
WebLLM q4f16_1 I	GSM8K	c2w=19, w2c=6	0.0146	9	0.2049	No
Native q4f16	MMLU	c2w=10, w2c=25	0.0167	10	0.2168	No
WebLLM q4f32_1 I	GSM8K	c2w=16, w2c=5	0.0266	11	0.3192	No
WebLLM q4f32_1 N	GSM8K	c2w=16, w2c=5	0.0266	12	0.3192	No
Native q4	MMLU	c2w=11, w2c=24	0.0410	13	0.4096	No
Native q4	GSM8K	c2w=13, w2c=4	0.0490	14	0.4414	No
WebLLM q4f16_1 N	GSM8K	c2w=16, w2c=6	0.0525	15	0.4414	No
WebLLM q4f32_1 I	MMLU	c2w=19, w2c=30	0.1524	16	1.0000	No
WebLLM q4f32_1 N	MMLU	c2w=19, w2c=30	0.1524	17	1.0000	No
Native q4f16	GSM8K	c2w=10, w2c=5	0.3018	18	1.0000	No
WebLLM q4f16_1 N	MMLU	c2w=22, w2c=30	0.3317	19	1.0000	No
WebLLM q4f16_1 I	MMLU	c2w=23, w2c=29	0.4885	20	1.0000	No
Native fp16	MMLU	c2w=2, w2c=0	0.5000	21	1.0000	No
Native fp16	GSM8K	c2w=2, w2c=4	0.6875	22	1.0000	No

References

[1] S. Niroula and M. Pokharel, "Does running a small LLM in the browser change its answers? Comparing native and WebGPU inference across GPUs (dataset and evaluation harness)," Zenodo, ver. 1.0.0, 2026, doi: 10.5281/zenodo.23760946.

[2] S. Niroula and M. Pokharel, "Browser LLM fidelity: Evaluation harness, archived outputs, and analysis pipeline," Zenodo, ver. 1.1.0, 2026, doi: 10.5281/zenodo.23828643. GitHub commit `f2e35b7`. [Online]. Available: https://github.com/imsudip45/browser-llm-fidelity/tree/f2e35b70009a23e25e72caa63c64494b4395b461

[3] Hugging Face, "Transformers.js documentation." [Online]. Available: https://huggingface.co/docs/transformers.js

[4] C. F. Ruan, Y. Qin, X. Zhou, et al., "WebLLM: A high-performance in-browser LLM inference engine," arXiv:2412.15803, 2024. [Online]. Available: https://arxiv.org/abs/2412.15803

[5] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee, "Accuracy is not all you need," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2407.09141

[6] R. Levine, R. Sharma, N. Jain, A. Ramesh, Z. Chen, N. Abbas, J. Contini, and T. Sorensen, "Llamas on the Web: Memory-efficient, performance-portable, and multi-precision LLM inference with WebGPU," arXiv:2605.20706, 2026. [Online]. Available: https://arxiv.org/abs/2605.20706

[7] J. Maczan, "Characterizing WebGPU dispatch overhead for LLM inference across four GPU vendors, three backends, and three browsers," arXiv:2604.02344, 2026. [Online]. Available: https://arxiv.org/abs/2604.02344

[8] G. Yang and F. Liu, "GenomeOcean Anywhere: Private WebGPU inference for genome MoEs," arXiv:2609.35882v1, Sep. 2026 (Withdrawn Oct. 1, 2026 pending institutional intellectual-property review). [Online]. Available: https://arxiv.org/abs/2609.35882

[9] A. Yang et al., "Qwen2.5 technical report," arXiv:2412.15115, 2024. [Online]. Available: https://arxiv.org/abs/2412.15115

[10] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *Proc. Int. Conf. Learn. Represent.*, 2021. [Online]. Available: https://arxiv.org/abs/2009.03300

[11] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021. [Online]. Available: https://arxiv.org/abs/2110.14168

[12] MLC-LLM Contributors, "MLC-LLM quantization registry," GitHub, revision `fcce2cc`. [Online]. Available: https://github.com/mlc-ai/mlc-llm/blob/fcce2cc3c88ca2f09c5743951ddc54474a690d6c/python/mlc_llm/quantization/quantization.py

[13] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[14] SciPy Community, "scipy.stats.binomtest (added in SciPy 1.7.0)," SciPy documentation. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.binomtest.html

[15] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.